Problem Set 1 - Suggested Solutions

Michaela Philip

March 5, 2025

Disclaimer: This code was written with the assistance of AI to interpret errors and troubleshoot.

1 Sum Stats and Institutional Details

1.1 Do some brief diligence on the products and industry. What do you anticipate may be some important determinants of demand, substitution, and pricing? (5 points)

5 points for reasonable answer

Answers may vary. Students may include information on conditions treated by the medications, the prevalence of these conditions, or relevant (perceived or otherwise) differences between the products. Responses may also include general information on popularity or use trends for the products.

1.2 Complete the table above by adding columns for the mean: market share, unit price, price/100 tablets, and unit wholesale price. Interpret any notable patterns you see in the summary statistics. (10 points)

5 points for discussion
5 points for accurate table

[28]:
```
import numpy as np
import pandas as pd
from linearmodels.iv import IV2SLS
import statsmodels.api as sm
import statsmodels.formula.api as smf
otc = pd.read_csv('OTC_Sales.csv')
```

[29]:
```
# calculate total sales for each store-week pair
product_numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]
otc['total_sales'] = otc[[f'sales_{number}' for number in product_numbers]].
 ↪sum(axis = 1)
```

[30]:
```
# reshape to make it easier to work with
otc = pd.wide_to_long(otc, ['sales', 'price', 'cost', 'prom'],
                      i = ['store', 'week'], j = 'product', sep = '_').
 ↪reset_index()
```

```
summary = otc.groupby('product').agg(
    price = ('price', 'mean'),
    total_sales = ('sales', 'sum'),
    wholesale_price = ('cost', 'mean')
).reset_index()

# add given information so that table is easy to read
brand_names = {
    1: 'Tylenol',
    2: 'Tylenol',
    3: 'Tylenol',
    4: 'Advil',
    5: 'Advil',
    6: 'Advil',
    7: 'Bayer',
    8: 'Bayer',
    9: 'Bayer',
    10: 'Store Brand',
    11: 'Store Brand'}
size = {
    1: 25,
    2: 50,
    3: 100,
    4: 25,
    5:  50,
    6: 100,
    7: 25,
    8: 50,
    9: 100,
    10: 50,
    11: 100
}
summary['brand'] = summary['product'].map(brand_names)
summary['size'] = summary['product'].map(size)

# calculate new variables
summary['market_share'] = summary['total_sales'] / (summary['total_sales'].
 ↪sum())
summary['price_per_100'] = (summary['price'] / summary['size']) * 100

# reorder columns and round for clarity
table_1 = summary[['product', 'brand', 'size', 'market_share',
                   'price', 'price_per_100', 'wholesale_price']].round(2)
table_1
```

[30]:

	product	brand	size	market_share	price	price_per_100	\
0	1	Tylenol	25	0.14	3.43	13.71	

1	2	Tylenol	50	0.18	4.95	9.89
2	3	Tylenol	100	0.12	7.03	7.03
3	4	Advil	25	0.12	2.97	11.88
4	5	Advil	50	0.08	5.15	10.29
5	6	Advil	100	0.04	8.16	8.16
6	7	Bayer	25	0.04	2.67	10.70
7	8	Bayer	50	0.03	3.62	7.24
8	9	Bayer	100	0.08	3.97	3.97
9	10	Store Brand	50	0.09	1.94	3.87
10	11	Store Brand	100	0.08	4.43	4.43

	wholesale_price
0	2.19
1	3.68
2	5.77
3	2.03
4	3.63
5	6.10
6	1.85
7	2.44
8	3.71
9	0.91
10	1.91

2 Logit Demand Estimation

Consider the utility function for product j in store-week t for consumer i:

$u_{ijt} = -\alpha p_{jt} + X_{jt}\beta + \xi_{jt} + \epsilon_{ijt}$ **(1)**

where p_{jt} is price, X_{jt} are other observed product characteristics, ξ_{jt} are unobserved product characteristics, and ϵ_{ijt} is an i.i.d. EV1 logit consumer-product-market unobservable.

Estimate this model:

2.1 Using OLS with price, promotion, and an indicator for whether the product is a "store brand" as product characteristics. (10 points)

5 points for correct setup
2 points for reassonable results in table
3 points for discussion

[31]:
```
# need shares for each observation
otc['share'] = otc['sales'] / otc['count']
otc['outside_share'] = 1 - (otc['total_sales'] / otc['count'])

# indicator for store brand
otc['store_brand'] = np.where(otc['product'] > 9, 1, 0)
```

```
otc['size'] = otc['product'].map(size)
```

[32]:
```
# create dependent variable: ln(share - outside_share)
otc['ln_share'] = np.log(otc['share']) - np.log(otc['outside_share'])
```

[33]:
```
model = smf.ols('ln_share ~ price + prom + store_brand', data = otc)
logit_modela = model.fit()
logit_modela.summary()
```

[33]:

Dep. Variable:	ln_share	**R-squared:**	0.026
Model:	OLS	**Adj. R-squared:**	0.026
Method:	Least Squares	**F-statistic:**	335.2
Date:	Wed, 05 Mar 2025	**Prob (F-statistic):**	8.60e-215
Time:	14:02:29	**Log-Likelihood:**	-48633.
No. Observations:	37290	**AIC:**	9.727e+04
Df Residuals:	37286	**BIC:**	9.731e+04
Df Model:	3		
Covariance Type:	nonrobust		

	coef	std err	t	P> |t|	[0.025	0.975]
Intercept	-7.6453	0.014	-552.117	0.000	-7.672	-7.618
price	-0.0662	0.003	-24.543	0.000	-0.071	-0.061
prom	0.1990	0.016	12.316	0.000	0.167	0.231
store_brand	-0.2672	0.013	-21.180	0.000	-0.292	-0.242

Omnibus:	2157.310	**Durbin-Watson:**	1.362
Prob(Omnibus):	0.000	**Jarque-Bera (JB):**	2189.616
Skew:	-0.552	**Prob(JB):**	0.00
Kurtosis:	2.563	**Cond. No.**	18.2

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

2.2 Using OLS with price and promotion as product characteristics and product fixed effects (where a "product" is a brand-size combination). (10 points)

5 points for correct setup
2 points for reasonable results in a table
3 points for discussion

[34]:
```
model = smf.ols('ln_share ~ price + prom + C(product)', data = otc)
logit_modelb = model.fit()
logit_modelb.summary()
```

[34]:

Dep. Variable:	ln_share	**R-squared:**	0.457
Model:	OLS	**Adj. R-squared:**	0.457
Method:	Least Squares	**F-statistic:**	2614.
Date:	Wed, 05 Mar 2025	**Prob (F-statistic):**	0.00
Time:	14:02:29	**Log-Likelihood:**	-37744.
No. Observations:	37290	**AIC:**	7.551e+04
Df Residuals:	37277	**BIC:**	7.562e+04
Df Model:	12		
Covariance Type:	nonrobust		

	coef	**std err**	**t**	**P> |t|**	**[0.025**	**0.975]**
Intercept	-6.0693	0.037	-164.587	0.000	-6.142	-5.997
C(product)[T.2]	0.6879	0.023	30.566	0.000	0.644	0.732
C(product)[T.3]	0.9188	0.040	22.778	0.000	0.840	0.998
C(product)[T.4]	-0.4312	0.017	-25.661	0.000	-0.464	-0.398
C(product)[T.5]	-0.1960	0.024	-8.178	0.000	-0.243	-0.149
C(product)[T.6]	0.0045	0.051	0.088	0.930	-0.096	0.105
C(product)[T.7]	-1.6597	0.018	-93.199	0.000	-1.695	-1.625
C(product)[T.8]	-1.5844	0.016	-96.398	0.000	-1.617	-1.552
C(product)[T.9]	-0.5523	0.018	-31.542	0.000	-0.587	-0.518
C(product)[T.10]	-1.2360	0.022	-56.168	0.000	-1.279	-1.193
C(product)[T.11]	-0.7387	0.019	-38.296	0.000	-0.777	-0.701
price	-0.3404	0.010	-33.375	0.000	-0.360	-0.320
prom	0.3220	0.013	25.508	0.000	0.297	0.347

Omnibus:	2680.344	**Durbin-Watson:**	1.901
Prob(Omnibus):	0.000	**Jarque-Bera (JB):**	3703.376
Skew:	-0.621	**Prob(JB):**	0.00
Kurtosis:	3.916	**Cond. No.**	112.

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

2.3 Estimate the models of (a) and (b) using the Hausman instrument (average price in other markets). (15 points)

5 points for correct calculation of Hausman instrument
3 points for correct 2SLS specification
2 points for reasonable results in table
5 points for discussion

[35]:
```
def get_instruments(df):
    # compute average prices per store, week, and product
    avg_prices = df.groupby(['store', 'week', 'product'])['price'].mean().
 ↪reset_index()

    # merge to bring average prices into the original dataframe
    df = df.merge(avg_prices, on=['store', 'week', 'product'], suffixes=('',␣
 ↪'_avg'))
```

```
    # compute instrument: average price across stores excluding the current␣
 ↪store
    df['instrument'] = df.groupby(['week', 'product'])['price_avg'].
 ↪transform(lambda x: (x.sum() - x) / (x.count() - 1))

    return df.drop(columns=['price_avg'])

otc = get_instruments(otc)
```

[36]:
```
# model a using IV
formula = ('ln_share ~ 1 + prom + store_brand + [price ~ instrument]')
logit_modela_iv = IV2SLS.from_formula(formula, otc).fit()
logit_modela_iv.summary
```

[36]:

Dep. Variable:	ln_share	**R-squared:**	0.0263
Estimator:	IV-2SLS	**Adj. R-squared:**	0.0262
No. Observations:	37290	**F-statistic:**	874.64
Date:	Wed, Mar 05 2025	**P-value (F-stat)**	0.0000
Time:	14:02:29	**Distribution:**	chi2(3)
Cov. Estimator:	robust		

	Parameter	Std. Err.	T-stat	P-value	Lower CI	Upper CI
Intercept	-7.6453	0.0143	-533.36	0.0000	-7.6734	-7.6172
prom	0.1990	0.0158	12.585	0.0000	0.1680	0.2300
store_brand	-0.2672	0.0136	-19.635	0.0000	-0.2938	-0.2405
price	-0.0662	0.0028	-23.364	0.0000	-0.0718	-0.0607

Endogenous: price
Instruments: instrument
Robust Covariance (Heteroskedastic)
Debiased: False

[37]:
```
# model b using IV
otc_p = pd.get_dummies(otc, columns = ['product'])
fixed_effects = " + ".join([col for col in otc_p.columns if col.
 ↪startswith("product_")])

formula = f'ln_share ~ prom  + {fixed_effects} + [price ~ instrument]'
logit_modelb_iv = IV2SLS.from_formula(formula, otc_p).fit()
logit_modelb_iv.summary
```

[37]:

Dep. Variable:	ln_share	**R-squared:**	0.4520
Estimator:	IV-2SLS	**Adj. R-squared:**	0.4519
No. Observations:	37290	**F-statistic:**	6.581e+06
Date:	Wed, Mar 05 2025	**P-value (F-stat)**	0.0000
Time:	14:02:30	**Distribution:**	chi2(13)
Cov. Estimator:	robust		

	Parameter	Std. Err.	T-stat	P-value	Lower CI	Upper CI
prom	0.2648	0.0142	18.676	0.0000	0.2370	0.2926
product_1	-5.4225	0.0531	-102.16	0.0000	-5.5265	-5.3184
product_2	-4.4459	0.0763	-58.246	0.0000	-4.5955	-4.2963
product_3	-3.8220	0.1081	-35.352	0.0000	-4.0338	-3.6101
product_4	-5.9368	0.0468	-126.82	0.0000	-6.0286	-5.8451
product_5	-5.2923	0.0797	-66.391	0.0000	-5.4485	-5.1361
product_6	-4.5235	0.1256	-36.002	0.0000	-4.7698	-4.2773
product_7	-7.2170	0.0432	-167.18	0.0000	-7.3016	-7.1324
product_8	-6.9629	0.0566	-122.93	0.0000	-7.0739	-6.8518
product_9	-5.8604	0.0629	-93.166	0.0000	-5.9837	-5.7371
product_10	-6.9324	0.0337	-205.62	0.0000	-6.9985	-6.8664
product_11	-5.9682	0.0683	-87.364	0.0000	-6.1021	-5.8343
price	-0.5286	0.0153	-34.607	0.0000	-0.5586	-0.4987

Endogenous: price
Instruments: instrument
Robust Covariance (Heteroskedastic)
Debiased: False

2.4 Compute the mean own-price elasticities for all products (10 points)

4 points for correct calculation of elasticities
2 points for reasonable results in table
4 points for discussion

[38]:
```
alpha = logit_modelb_iv.params.price
otc['elasticity'] = alpha * (otc['price']) * (1 - otc['share'])

# add this to our summary table so that we can easily compare
sum_elast = otc.groupby('product').agg(elasticity = ('elasticity', 'mean')).
 ↪reset_index()
summary = pd.merge(summary, sum_elast, how = 'outer')
summary.round(3)
```

[38]:

	product	price	total_sales	wholesale_price	brand	size	\
0	1	3.427	51280	2.187	Tylenol	25	
1	2	4.946	63604	3.681	Tylenol	50	
2	3	7.028	41305	5.771	Tylenol	100	
3	4	2.969	42032	2.027	Advil	25	

4	5	5.146	27627	3.630	Advil	50
5	6	8.158	12703	6.104	Advil	100
6	7	2.674	14365	1.851	Bayer	25
7	8	3.619	12310	2.438	Bayer	50
8	9	3.971	28324	3.710	Bayer	100
9	10	1.935	33175	0.910	Store Brand	50
10	11	4.427	27208	1.914	Store Brand	100

	market_share	price_per_100	elasticity
0	0.145	13.708	-1.810
1	0.180	9.893	-2.612
2	0.117	7.028	-3.713
3	0.119	11.876	-1.568
4	0.078	10.291	-2.719
5	0.036	8.158	-4.312
6	0.041	10.697	-1.413
7	0.035	7.238	-1.913
8	0.080	3.971	-2.098
9	0.094	3.871	-1.023
10	0.077	4.427	-2.339

3 New Product Introduction

The chain of stores you have data from is considering introducing a 25 tablet size bottle.

3.1 Assume WTP for the new product will be equal to average WTP of the brand name 25 tablet products, minus the "store brand" effect you estimated in the first part above. Assume there are no promotions of the new product. What will be the expected demand for the new product at a price of $2.00? (15 points)

5 points for correct calculation of WTP
56 points for correct calculation of new product share
2 points for reasonable results
3 points for discussion

[39]:
```
# WTP = average product_fe - store_brand
beta_product = {}
for i in product_numbers:
    beta_product[i] = logit_modelb_iv.params.get(f"product_{i}", 1)
beta_fe = pd.Series(beta_product)

beta_storebrand = logit_modela_iv.params.store_brand

WTP = ((beta_product[1] + beta_product[4] + beta_product[7])/3) -␣
 ↪beta_storebrand
```

[40]:
```
# pull coefficients for use in calculating shares
beta_prom = logit_modelb_iv.params.prom
alpha = logit_modelb_iv.params.price

beta_product = {}
for i in product_numbers:
    beta_product[i] = logit_modelb_iv.params.get(f"product_{i}", 1)

beta_fe = pd.Series(beta_product)
```

[41]:
```
# recalculate market shares with new good
# calculate logit share numerator for each good
numerator = {}
for i in product_numbers:
    otc_i = otc[otc['product'] == i]
    numerator[i] = np.exp((alpha * otc_i['price'].mean()) + (beta_prom *␣
 ↪otc_i['prom'].mean()) +
                          (beta_storebrand * otc_i['store_brand'].mean()) +␣
 ↪beta_fe[i])
numerator_before_new_good = pd.Series(numerator)
numerator_after_new_good = pd.Series(numerator)

# add share of new good using WTP calculated above as product FE
numerator_after_new_good[12] = np.exp((alpha * 2) + (beta_prom * 0) +␣
 ↪(beta_storebrand * 1) + WTP)

# calculate denominators for before and after new good is introduced
denominator_before_new_good = 1 + numerator_before_new_good.sum()
denominator_after_new_good = 1 + numerator_after_new_good.sum()
```

[42]:
```
# calculate total customers across all stores in all weeks
total_traffic = (otc.groupby(['store', 'week'])['count'].first()).sum()

# estimate sales of each good before and after new product introduction
new_market_shares = numerator_after_new_good / denominator_after_new_good
sales_new = new_market_shares * total_traffic
old_market_shares = numerator_before_new_good / denominator_before_new_good

# add in value of 0 for new product before it was introduced
old_market_shares[12] = 0
sales_old = old_market_shares * total_traffic
```

[43]:
```
# put it all into a table
otc_new = pd.DataFrame({
    'Old Shares': old_market_shares.values,
    'Old Sales': sales_old.values.round(2),
    'New Shares': new_market_shares.values,
```

```
    'New Sales': sales_new.values.round(2)},
    index = new_market_shares.index
)

# rename the index to 'Products'
otc_new.index.name = 'Products'
otc_new
```

[43]:

Products	Old Shares	Old Sales	New Shares	New Sales
1	0.000725	48283.29	0.000724	48249.15
2	0.000873	58139.10	0.000872	58098.00
3	0.000545	36317.32	0.000545	36291.64
4	0.000560	37299.86	0.000559	37273.48
5	0.000337	22447.71	0.000337	22431.84
6	0.000149	9907.89	0.000149	9900.88
7	0.000185	12334.48	0.000185	12325.76
8	0.000146	9695.65	0.000145	9688.80
9	0.000371	24705.69	0.000371	24688.23
10	0.000278	18531.23	0.000278	18518.13
11	0.000194	12915.60	0.000194	12906.47
12	0.000000	0.00	0.000707	47109.12

3.2 Break down the benefits and costs to the store of introducing this new format (assume wholesale price is $1.00 and there are no fixed or other variable costs of the new product introduction). (10 points)

6 points for correct calculation of new market shares
2 points for reasonable results
2 points for discussion

[44]:
```
# pull price and cost information and add in the new product
price = otc.groupby('product')['price'].mean()
cost = otc.groupby('product')['cost'].mean()
price[12] = 2
cost[12] = 1

# add to otc_new
otc_new['Price'] = price.values.round(2)
otc_new['Cost'] = cost.values.round(2)
```

[45]:
```
# calculate profit before and after the new product
otc_new['Old Profit'] = ((otc_new['Price'] - otc_new['Cost']) * (otc_new['Old␣
 ↪Sales'])).round(2)
otc_new['New Profit'] = ((otc_new['Price'] - otc_new['Cost']) * (otc_new['New␣
 ↪Sales'])).round(2)
```

```
print(f'Profit before the new product: {otc_new['Old Profit'].sum().round(2)}')
print(f'Profit after the new product: {otc_new['New Profit'].sum().round(2)}')
print(f'Therefore, introducing the new product changes profit by␣
 ↪{((otc_new['New Profit'].sum()) - (otc_new['Old Profit'].sum())).round(2)}')
```

```
Profit before the new product: 348673.5
Profit after the new product: 395536.09
Therefore, introducing the new product changes profit by 46862.59
```

4 Information Intervention

The chain of stores you have data from is considering a campaign to help educate customers that there is no efficacy difference between the brand name and "store brand" drugs.

4.1 Assume the campaign increases the WTP for the "store brands" by the full absolute value of the "store brand" effect you estimated in the first part. What is the net benefit and cost to the store? To consumers? (15 points)

5 points for correct calculation of WTP
5 points for correct calculation of new shares
2 points for reasonable results
3 points for discussion

[46]:
```
# pull fe again (just in case)
beta_product = {}
for i in product_numbers:
    beta_product[i] = logit_modelb_iv.params.get(f"product_{i}", 1)
beta_fe = pd.Series(beta_product)

beta_storebrand = logit_modela_iv.params.store_brand
```

[47]:
```
# calculate CS before change
# calculate logit share numerator for each good (should be the same)
numerator = {}
for i in product_numbers:
    otc_i = otc[otc['product'] == i]
    numerator[i] = np.exp((alpha * otc_i['price'].mean()) + (beta_prom *␣
 ↪otc_i['prom'].mean())
                          + beta_fe[i] + ((beta_storebrand) *␣
 ↪otc_i['store_brand'].mean()))

numerator_before = pd.Series(numerator)
denominator_before = 1 + numerator_before.sum()

# need to artificially increase the utility received by patients buying␣
 ↪store-brand drugs
```

```
adjusted_denominator_before = denominator_before + (np.exp(np.
 ↪abs(beta_storebrand) * 2))
```

[48]:
```
# recalculate WTP without the store-brand effect
beta_fe[10] = beta_fe[10] + np.abs(beta_storebrand)
beta_fe[11] = beta_fe[11] + np.abs(beta_storebrand)
```

[49]:
```
# recalculate market shares with new WTP
# calculate logit share numerator for each good
numerator = {}
for i in product_numbers:
    otc_i = otc[otc['product'] == i]
    numerator[i] = np.exp((alpha * otc_i['price'].mean()) + (beta_prom *␣
 ↪otc_i['prom'].mean())
                          + beta_fe[i])

numerator_after = pd.Series(numerator)
denominator_after = 1 + numerator_after.sum()
```

[50]:
```
# estimate sales of each good before and after information campaign
new_market_shares = numerator_after / denominator_after
sales_new = new_market_shares * total_traffic
old_market_shares = numerator_before / denominator_before
sales_old = old_market_shares * total_traffic
```

[51]:
```
# put it all into a table
otc_info = pd.DataFrame({
    'Old Shares': old_market_shares.values,
    'Old Sales': sales_old.values.round(2),
    'New Shares': new_market_shares.values,
    'New Sales': sales_new.values.round(2)},
    index = new_market_shares.index
)

# Rename the index to 'Products'
otc_info.index.name = 'Products'
otc_info
```

[51]:

Products	Old Shares	Old Sales	New Shares	New Sales
1	0.000725	48283.29	0.000724	48267.20
2	0.000873	58139.10	0.000872	58119.73
3	0.000545	36317.32	0.000545	36305.22
4	0.000560	37299.86	0.000560	37287.43
5	0.000337	22447.71	0.000337	22440.23
6	0.000149	9907.89	0.000149	9904.59
7	0.000185	12334.48	0.000185	12330.37

```
8           0.000146    9695.65    0.000145    9692.42
9           0.000371   24705.69    0.000371   24697.46
10          0.000278   18531.23    0.000474   31609.69
11          0.000194   12915.60    0.000331   22030.82
```

[52]:
```
# pull price and cost information and add in the new product
price = otc.groupby('product')['price'].mean()
cost = otc.groupby('product')['cost'].mean()

# add to otc_info
otc_info['Price'] = price.values.round(2)
otc_info['Cost'] = cost.values.round(2)
```

[53]:
```
# calculate profit before and after the new product
otc_info['Old Profit'] = ((otc_info['Price'] - otc_info['Cost']) *␣
 ↪(otc_info['Old Sales'])).round(2)
otc_info['New Profit'] = ((otc_info['Price'] - otc_info['Cost']) *␣
 ↪(otc_info['New Sales'])).round(2)

print(f'Profit before the new product: {otc_info['Old Profit'].sum().round(2)}')
print(f'Profit after the new product: {otc_info['New Profit'].sum().round(2)}')
print(f'Therefore, introducing the new product changes profit by␣
 ↪{((otc_info['New Profit'].sum()) - (otc_info['Old Profit'].sum())).
 ↪round(2)}')
```

```
Profit before the new product: 348673.5
Profit after the new product: 385015.71
Therefore, introducing the new product changes profit by 36342.21
```

[54]:
```
# CS = log(denominator)/alpha
CS_before = np.log(adjusted_denominator_before) / alpha
CS_after = np.log(denominator_after) / alpha

print(f'Consumer Surplus before the information campaign is {CS_before}')
print(f'Consumer Surplus after the information campaign is {CS_after}')
print(f'Consumer Surplus increases by {(CS_after - CS_before)}')
```

```
Consumer Surplus before the information campaign is -1.886442797688499
Consumer Surplus after the information campaign is -0.008898765290945184
Consumer Surplus increases by 1.8775440323975539
```